Solution: Buildings With an Ocean View

Problem Statement

You are given an array `heights` of size `n`, representing the heights of `n` buildings.

The `ocean` is to the `right` of these buildings. A building has an ocean view if there are no `taller` buildings to its right.

The task is to find the `indices(0-based)` of all such buildings with an ocean view.

Examples

- **Example 1**:
  - **Input**: `[4, 3, 2, 1]`
  - **Expected Output**: `[0, 1, 2, 3]`
  - **Justification**: Each building is shorter than the one to its left, hence all have an ocean view.
- **Example 2**:
  - **Input**: `[2, 3, 1, 4]`
  - **Expected Output**: `[3]`
  - **Justification**: Building at index 3 is the only one without taller buildings to their right.
- **Example 3**:
  - **Input**: `[7, 4, 3, 2, 1, 4, 6, 3]`
  - **Expected Output**: `[0, 6, 7]`
  - **Justification**: Buildings at indices 0, 6, and 7 are the only ones without taller buildings to their right.

Solution

To solve this problem, the most effective approach is to traverse the array of buildings from right to left, maintaining a record of the tallest building encountered so far. This direction of traversal is crucial because it allows us to easily determine whether a building has a taller one to its right. By keeping track of the maximum height encountered, we can identify which buildings have an unobstructed view of the ocean.

The reason why this approach is efficient lies in its simplicity and the reduction of unnecessary comparisons. Instead of comparing each building with all others to its right, we only compare it with the maximum height encountered so far. This significantly reduces the number of comparisons and makes the algorithm efficient for large lists of buildings.

Step-by-step Algorithm

- Initialize an empty list `result` to store indices of buildings with ocean views.
- Initialize a variable `maxHeight` to 0 to keep track of the tallest building seen so far.
- Traverse the array from the last element to the first:
  - If the current building's height is greater than `maxHeight`:
    - Add the current index to `result`.
    - Update `maxHeight` to the current building's height.
- Reverse the `result` list to maintain the original order of buildings.
- Return the `result` list.

Algorithm Walkthrough

Consider the input `[7, 4, 3, 2, 1, 4, 6, 3]`.

- **Step 1**: Initialize `result = []`, `maxHeight = 0`.
- **Step 2**: Start from the last building (height 3). `3 > 0`, so add index `7` to `result`, update `maxHeight` to `3`. result = `[7]`.
- **Step 3**: Move to the second-last building (height 6). `6 > 3`, so add index `6` to `result`, update `maxHeight` to `6`. result = `[7, 6]`.
- **Step 4**: Move to the index `5`. building height = 4. `4 < 6`, do nothing.
- **Step 5**: Move to the index `4`. building height = 1. `1 < 6`, do nothing.
- **Step 6**: Move to the index `3`. building height = 2. `2 < 6`, do nothing.
- **Step 7**: Move to the index `2`. building height = 3. `3 < 6`, do nothing.
- **Step 8**: Move to the index `1`. building height = 4. `4 < 6`, do nothing.
- **Step 9**: Move to the index `0`. building height = 7. `7 > 6`, so add index `0` to `result` and update `maxHeight` to `7`. result = `[7, 6, 0]`.
- **Step 7**: Reverse `result` to `[7, 6, 0]`.
- **Final Output**: `[0, 6, 7]`.

Code

Java

```java
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Solution {
    public List<Integer> findBuildings(int[] heights) {
        List<Integer> result = new ArrayList<>();
        int maxHeight = 0; // Initialize the maximum height encountered so far

        // Traverse the array from the last element to the first
        for (int i = heights.length - 1; i >= 0; i--) {
            if (heights[i] > maxHeight) {
                result.add(i); // Add index of the building with ocean view
                maxHeight = heights[i]; // Update the maximum height
            }
        }
        Collections.reverse(result); // Reverse the list to maintain original order
        return result;
    }

    public static void main(String[] args) {
```

Run Code Result Stdout

You must run your code first

Execute

Complexity Analysis

- **Time Complexity**: The time complexity of the algorithm is `O(N)`, where N is the number of buildings. This is because we traverse the list of buildings once from right to left.
- **Space Complexity**: The space complexity is `O(N)` in the worst case, where all buildings have an ocean view. This space is used to store the indices of buildings in the result list. In practical scenarios, the space used would be less than `O(N)` as not all buildings will have an ocean view.

← Previous
Buildings With An Ocean View (Medium)

Next →
Basic Calculator II (Medium)

☐ Mark as Completed